

数据库(11)

孙甲松

sunjiasong@tsinghua.edu.cn

电子工程系 信息认知与智能系统研究所

罗姆楼6-104

电话: 13901216180/62796193

2025.11.

第7章 数据库保护

7.3 并发控制(Concurrency Control)

- 数据库是共享资源，可以由多个用户同时使用。
- 一般情况下，这些用户的操作应串行执行，每一时刻只有一个用户程序运行。
- 若不加限制很可能会破坏数据库的完整性(Integrity) 和一致性(Consistency)。

7.3 并发控制 (Concurrency Control)

目录

7.3.1 几个基本概念

7.3.2 封锁

7.3.3 活锁和死锁

7.3.4 可串行化

7.3.5 两阶段锁协议

7.3.6 封锁的粒度

7.3.7 Oracle的数据库并发控制

7.3.8 高并发情况下的数据准确性

7.3.1 几个基本概念

(一) 事务

定义：事务（Transaction）是并发控制的单位。所谓事务是一个数据库操作序列，这些操作要么都做，要么都不做，是一个不可分割的工作单元。

- 事务的4个特性：

- 原子性 Atomicity 不可分割，都做/都不做
- 一致性 Consistency 执行保持数据一致性状态
- 隔离性 Isolation 执行过程不受其它干扰
- 持续性 Durability 对数据改变的永久性

- 简称为：事务的ACID特性。

- SQL查询分析器的每次提交都是一个事务。

7.3.1 几个基本概念

- 同一个数据库程序可以做为几个事务提交：

BEGIN Transaction

.....

COMMIT

} 为一个事务

BEGIN Transaction

.....

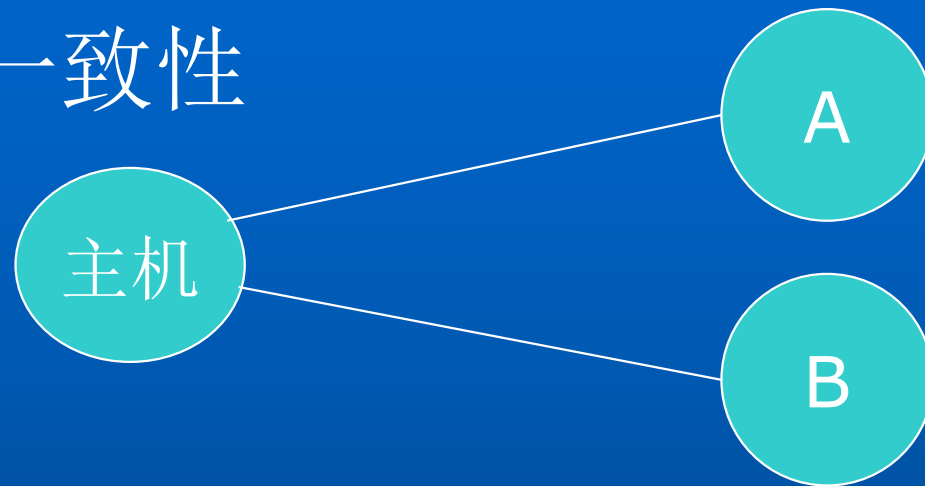
COMMIT

} 为另一个事务

- 事务和程序是两个不同的概念，与进程和程序关系相似。
- 一个程序可包括多个事务（类似于一个程序可以启动多个进程）。

7.3.1 几个基本概念

(二) 数据一致性



A、B是售票点，A查询有10张票，与此同时B查询有10张票，A卖出1张，将主机改为9，与此同时B卖出1张，将主机改为9。卖出2张只记录1张！

- 可能产生的几种不一致性：
 - 丢失修改，上面的例子
 - 不可重复读（另一个事务修改删除插入引起）
 - 读“脏”数据，无效数据

T95次列车同一车厢12个铺位出现重票

<http://www.sina.com.cn> 2008年02月16日14:15 武汉晚报

本报讯 (记者 高星) 拿着火车票找到自己的铺位，却发现该铺上已经有2个乘客在争执了，3人大眼瞪小眼：手中拿的车票竟然一模一样！列车长闻讯赶来查看，票还都是真的！昨日下午，坐火车遇到蹊跷事的王先生向本报求助。

昨日下午6时许，王先生乘上由武昌发往深圳的T95次列车，来到9号车厢的19号上铺时，竟有3名乘客同时持有该铺位的车票：王先生是昨日在车站售票窗口买的；另一位香港的旅客是在深圳订的返程票；还有一乘客是提前数天托人买的票。不仅如此，该车厢17、18、19、20号（上铺、中铺、下铺）共计12个铺位都出现了重票，13位乘客有票无铺。

昨日，记者联系上该车车长，他表示**乘客所持车票确实都是真的**，他们在尽力给乘客进行安排，目前采取的措施是将下行站点预留的铺位腾出来，再与下行站点做好沟通和协调。昨晚6点18分，王先生再次致电本报表示，8名乘客已安排到铺位，另5名乘客的铺位，列车长正在积极安排。

记者昨查阅相关资料发现，广州开往沈阳的T11次、赣州开往北京西的1626次、京津城际特快等列车上都出现过重票问题，铁路部门的解释是**计算机票务系统出现故障，不同售票点出了同一张车票**，但这种情况比较罕见。

订单详情

下单时间:2023.08.08

18:32

19:02

红停站

历时30分

发车时间: 2023.08.11 星期五

车票——八、四、二、一、X

检票口2 (如有变更, 请以现场公告为准)

变更到站

改签

退票

靠窗 二等座 08车 15A号

中国姓氏之考证

已进站

退改说明

订餐

退乘意險

二等座 08车 15B号>

中国居民身份证



啵啵杮

关注

坐高铁重票了👁👁

今天晚上坐高铁从某站到终点站(也就半个小时)，然后上车发现自己的座位上有人🤔，对了好几遍票，还以为是和之前很多人遇到的情况一样(对方没有在官方APP上买票)，结果发现好几个和我们一站上车的人座位上也有人。坐在和我们重的座位上的人说他们是从起始站到终点站的票，然后我们把乘务员姐姐叫了过来，她验了我们所有人的票，发现我们的票都是有效的，也都没有坐错车🤔。乘务员姐姐也不知道为什么，然后她把我们全都带到了有座位的地方。真的太离谱了。🤔🤔🤔#高铁票重票

2023-08-11



7.3.1 几个基本概念

- **并发控制**就是要用正确的方式调度并发操作，避免造成数据的修改丢失或不一致性。
- 主要方法有：
 - (1) 封锁 (locking)
 - (2) 时间印 或称为：时间戳 (time stamp)
 - (3) 乐观控制法 (optimistic scheduler)
 - (4) 多版本并发控制 (multi-version concurrency control, 简称 MVCC)
- 下面介绍**封锁**技术，现在绝大部分系统都采用封锁方法。

7.3.2 封锁

- 封锁（Locking）是事务在对某个数据进行操作之前，先向系统发出请求，对要操作的数据对象加锁。
- 在事务A对数据对象B的操作完成之前，封锁B，其它事务必须等事务A完成对B的操作并开锁后，才能访问数据对象B。

Lock---Unlock

- 基本的封锁类型有：
 - { 排他锁（eXclusive Locks） 简称：X锁
 - { 共享锁（Share Locks） 简称：S锁

7.3.2 封锁

- 假定有事务T，数据A：
- ①若T对A加X锁，则只允许T读或修改A，其他一切事务对A的任何封锁请求都不能成功，直到T释放（打开）A的X锁。又称为：写锁
- ②若T对A加S锁，则其他一切事务对A的X锁请求不能成功，而对A的S锁请求可以。即其他事务可以读A，但不能修改A，直到T释放（打开）A上的S锁。又称为：读锁
- 因此，同一个资源可同时被加上个多个S锁。
- 利用X锁和S锁,可以建立不同级别的封锁协议。
- 三级协议：①加X锁 ②加S锁 ③加X锁并加S锁

7.3.2 封锁 三级封锁协议

①加X锁

事务T独享数据A，可查看并修改，别的事务因为不能对A加X锁或S锁，连查看数据A的权限都没有。但一旦X锁解除，别的事务可以对数据A加X锁或S锁，对数据A进行修改或查看。可防止丢失修改。

②加S锁

T对A加S锁，则其他一切事务只能对A加S锁，同一个数据A上可能会被加上多个S锁，所有事务可以读A，但不能修改A，直到所有A上的S锁被解除后，才能继续加X锁并修改。可进一步防止读“脏”数据。

③加X锁并加S锁

事务T独享数据A，可查看并修改，通过加S锁，在X锁解除后，允许别的事务查看数据A，但始终不允许别的事务修改数据A。可更进一步防止不可重复读。

7.3.3 活锁和死锁

➤活锁:

如果事务T1封锁了数据R，此时事务T2也申请封锁R，T2等待；若事务T3也申请封锁R，在T1释放了R上的封锁后，首先批准了T3的封锁请求，T2继续等待；若此时事务T4也申请封锁R，在T3释放了R上的封锁后，首先批准了T4的封锁请求，T2仍然需要等待；.....若T2运气不好，有可能一直等待下去，形成活锁。

➤避免活锁的简单方法:

先来先服务的策略，当多个事务请求封锁同一个数据时，按请求的先后顺序对这些事务排队，每次排在列首的事务获得封锁权。这样可以避免出现某个事务因为各种巧合无法获得封锁权一直等待的活锁情形。

7.3.3 活锁和死锁

➤ 死锁 (dead lock) :

T1—^x→ R1 等R2

T2—^x→ R2 等R3

T3—^x→ R3 等R1 形成死锁！

➤ 因此为避免死锁，要求：

- ① 一次封锁法 要求每个事务一次就将所有要使用的数据全部加锁，否则一个都不加。
- ② 顺序封锁法 预先规定一个封锁顺序，所有事务按此顺序加锁。
- ③ 死锁的诊断与解除法 不预防，但先诊断。比如有：超时法，等待图法，发生死锁则设法解除。比如，撤销其中一个处理死锁代价最小的事务，释放此事务持有的所有加锁，打破怪圈，解除死锁。



广西新闻网 www.gxnews.com.cn
广西重点新闻网站



7.3.4 可串行性

- **定义** 当且仅当某组事务的一定交叉调度产生的结果 和 这些事务的某一串行调度的结果相同，则这个交叉调度是**可串行化的调度**。
- **可串行性**（Serializability）是并行执事务正确性的准则。
- 为了保证并发操作的正确性，DBMS的并发控制机制必须提供一定的手段来保证调度是可串行化的。
- 主要采用封锁方法：**两段锁协议**
- 还有**时间印**（**时间戳**，Time-Stamping）方法、乐观方法等。

7.3.5 两段锁协议

- **两段锁协议**(Two-Phase Locking Protocol):
- 所谓两段锁协议是指所有事务必须分两个阶段对数据项加锁和解锁:
- 1. 在对任何数据进行读、写之前, 首先要获得对该数据的封锁。
- 2. 在释放一个封锁之后, 事务不再申请和获得任何其他封锁。
- 事务执行分为两个阶段:
- 第一阶段: 获得封锁, 扩展阶段
- 第二阶段: 释放封锁, 收缩阶段
- 注意: 开始释放封锁就不能再申请新的封锁!

7.3.5 两段锁协议

- 定理：若并发执行的所有事务均遵守两段锁协议，则对这些事务的任何并发调度策略都是可串行化的。
- 但反过来，若某并发事务的一个调度策略是可串行化的，不一定所有事务都遵守两段锁协议。

7.3.5 两段锁协议

➤ 两段锁协议与预防死锁的一次封锁法相比：

1. 一次封锁法要求每个事务必须一次将所有要使用的数据全部加锁，否则不能继续执行。因此一次封锁法遵守两段锁协议。

2. 但两段锁协议并不要求事务必须一次将所有要使用的数据全部加锁，可以分批分次加锁，因此即使是遵守两段锁协议的事务仍可能发生死锁！

3. 因此DBMS必须还要有检测机制，解除死锁。比如超时法、等待图法等。

7.3.6 封锁粒度

- 封锁对象的大小称为封锁粒度（granularity），直接影响系统的效率：

{ 单元小，并发度高，封锁机构复杂，系统开销大
单元大，并发度低，封锁机构简单，等待时间长

- 封锁对象可以是属性值，元组，关系，数据库等。
- 必须针对不同类型数据库选择合适的封锁粒度，以求最优的效果。
- 如果在一个系统中同时支持多种封锁粒度供不同的事务选择，这种封锁方法称为多粒度封锁（multiple granularity locking）。

7.3.6 封锁粒度

- 选择封锁粒度时应该同时考虑封锁开销和并发度两个因素，适当选择封锁粒度以求得最优的效果。一般说来：
- 需要处理大量元组的事务可以以关系为封锁粒度。
- 需要处理多个关系的大量元组的事务可以以数据库为封锁粒度。
- 而对于一个处理少量元组的用户事务，以元组为封锁粒度比较合适。
- 也可以混合，多粒度封锁，比如：

数据库-关系-元组

建立多粒度树，树的根结点是整个数据库，多粒度封锁协议允许对树中的每个结点独立加锁，引出了：**显式封锁** 和 **隐式封锁**（上一级结点加锁使得其加锁）。

7.3.6 封锁粒度

➤ 意向锁

如果对一个结点加意向锁，则说明该结点的下层结点正在被加锁；对任一结点加锁时，必须先对它的上层结点加意向锁。

例如：对任一个元组加锁时，必须先对元组所在的数据库和关系加意向锁。

➤ IS锁 意向共享锁(Intent Share lock)

如果对一个数据对象加IS锁，表示它的子结点拟(意向)加S锁。

例如：若事务T1要对关系R1的某个元组加S锁时，则首先对关系R1和R1所在的数据库加IS锁。

➤ IX锁 意向排他锁(Intent eXclusive lock)

如果对一个数据对象加IX锁，表示它的子结点拟(意向)加X锁。

例如：若事务T1要对关系R1的某个元组加X锁时，则首先对关系R1和R1所在的数据库加IX锁。

7.3.6 封锁粒度

➤ SIX锁 共享意向排他锁(Share Intent eXclusive lock)

如果要对一个数据对象加SIX锁，表示对它加S锁，再加IX锁，即： $SIX = S + IX$ 。

例如：若事务T1要对关系R1加SIX锁，则表示该事务首先要读整个关系R1，因此先对该关系R1加S锁，同时会更新关系R1中个别元组，因此要对关系R1加IX锁。

➤ S锁，X锁，IS锁，IX锁，SIX锁这5种锁之间有相容性和不相容性。

参见参考书第5版322页图11.11，第6版348页图12.12中 (a) 封锁类型的相容矩阵。加锁时，需要检查是否有不相容的锁存在。

例如，若事务T1要对关系R1加S锁，则要首先对R1所在的数据库加IS锁。这需要先检查R1所在的数据库和R1是否已加了不相容的X或IX锁，但不再需要搜索和检查R1中的元组是否加了不相容的X锁。

T_1	T_2					
	S	X	IS	IX	SIX	—
S	Y	N	Y	N	N	Y
X	N	N	N	N	N	Y
IS	Y	N	Y	Y	Y	Y
IX	N	N	Y	Y	N	Y
SIX	N	N	Y	N	N	Y
—	Y	Y	Y	Y	Y	Y

注：Y=Yes,表示相容的请求;N=No,表示不相容的请求

(a) 封锁类型的相容矩阵

7.3.7 Oracle的数据库并发控制

- Oracle采用封锁技术。
- Oracle的锁分为两大类：数据锁和字典锁。
- Oracle的数据锁（DML锁）包括5种：
 - 共享锁(S锁)
 - 排他锁(X锁)
 - 行级共享锁(RS锁) IS锁 意向共享锁
 - 行级排他锁(RX锁) IX锁 意向排他锁
 - 共享行级排他锁(SRX锁)
 - SRX锁 共享意向排他锁
- Oracle的封锁粒度包括：行级和表级
- 读数据时不加锁，通过回滚段保证可重复读。

7.3.8 高并发情况下的数据准确性

➤ 高并发情况下的数据准确性

随着数据库项目的推广及数据量的增长，用户数量也会随之增加，同时带来的高并发对数据准确性的考验逐渐增强。系统能够保证万级并发时数据的准确性，比如大量大学生同时预约同一导师的咨询，绝不可产生数据的错误与丢失。为应对此类并发问题采用了Redis原子操作，对高并发数据进行监控处理。Redis原子操作是很多秒杀项目中常用的技术，对高并发的处理十分出色。

● Redis简介

Redis 是完全开源免费的，遵守BSD协议，是一个基于内存的高性能的key-value数据库。

● Redis 的三个特点:

(1) Redis支持数据的持久化, 可以将内存中的数据保存在磁盘中, 重启的时候可以再次加载进行使用。

(2) Redis提供了丰富的类型, 如string, list, set, zset, hash共五种数据结构, 都是key-value的结构。

(3) Redis支持数据的备份, 即master-slave模式的数据备份。

● Redis 的优势

➤ 性能极高 – Redis读的速度是110000次/s, 写的速度是81000次/s。

➤ 丰富的数据类型 – Redis支持二进制案例的 Strings, Lists, Hashes, Sets 及 Ordered Sets 数据类型操作。

➤ 原子 – Redis的所有操作都是原子性的, 意思就是要么成功执行要么失败完全不执行。单个操作是原子性的。多个操作也支持事务, 即原子性, 通过MULTI和EXEC指令包起来。

➤ 丰富的特性 – Redis还支持 publish/subscribe, 通知, key 过期等特性。有兴趣同学可参见:

https://blog.csdn.net/weixin_39792747/article/details/110515955

7.4 数据库恢复(Recovery)

- 系统出现：硬件故障、软件错误、操作失误、人为故意破坏等导致数据库被损坏，如何恢复。
- DBMS必须具有把数据库从错误状态恢复到某一已知正确状态(一致状态或完整状态)的功能，这个功能就是数据库的恢复功能。
- DBMS的恢复功能是整个系统的一个重要组成部分，恢复功能部分的代码往往占整个系统代码的十分之一以上。
- 恢复功能是衡量DBMS好坏的一个重要指标。

7.4 数据库恢复(Recovery)

目 录

7.4.1 故障的种类

7.4.2 转储和恢复

7.4.3 日志文件

7.4.4 实际系统采用的恢复技术

7.4.1 故障的种类

1. 系统内部的故障

例如：转帐时的空头支票。可以通过强行回滚（ROLLBACK）进行事务撤销（UNDO）。将未执行完的事务撤销，恢复原状。

2. 系统故障

- CPU故障、OS（操作系统）故障、DBMS错误、停电等突发事件。被称为软故障（Soft Crash）。
- 影响正在运行的事务，但一般不破坏数据库。
- 由于事务未执行完，数据库处于错误状态。
- 为保证数据一致性，需要清除事务对数据已做的修改，把数据库恢复原状。
- 使用强行回滚（UNDO）。

7.4.1 故障的种类

- 再使用重做（REDO）命令将已经提交的事务都执行，使得数据库真正恢复到一致状态，彻底消除系统故障造成的影响。

3. 介质故障

- 通常指存储介质损坏，比如硬盘损坏。又称为硬故障(Hard Crash)，对应软故障。
- 这类故障破坏性大，不好恢复。
- 除非数据有备份：定期复制的CD、DVD、Tape，或镜像备份、RAID5阵列、双机热备份。

4. 计算机病毒

- 种类很多，变化很快。需要定期杀毒。

7.4.2 转储和恢复

- 转储即做备份，是由DBA定期做。
- 为数据做副本，每天或每周刻录CD、DVD，复制到Tape、移动硬盘等上面，这被称为**静态转储**。
- 利用静态转储的数据进行恢复，只能恢复到备份时的状态，备份以后的数据不能恢复。
- 利用双硬盘的镜像、双机热备份或RAID5磁盘阵列系统等被称为**动态转储**。可以保证数据每个时刻都有备份。
- 双机热备份的计算机系统一般分开独立存放，不能把全部鸡蛋放在同一个篮子里。
- 利用动态转储基本能恢复数据库的全部数据。
- 甚至在发生故障时基本不影响当前系统的使用。

7.4.3 日志文件

- 日志文件（Log File）用来记录对数据库的每一次更新操作。
- 当出现问题时，利用日志文件可以完全恢复数据库原状。

1. 登录日志文件

- 把每一个事务的每一个插入、删除、修改操作记录下来，包括操作之前之后的值。
- 必须严格按照事务的执行顺序登录，且按照先登录日志文件，再访问数据库的原则。

2. 事务恢复

- 按日志文件由DBMS或DBA自动重新执行。

还有一类审计日志文件（Audit Log File）用来记录对数据库的任何操作，包括读、写、修改。事无巨细。

7.4.4 实际系统采用的恢复技术

- Oracle 7采用了转储和登录日志文件技术。
 - 提供了REDO日志文件和回滚段，能有效恢复数据。
 - 当出现问题时，利用日志文件可以完全恢复数据库原状。
-
- SQL Server 采用了镜像和日志文件技术。
 - 安装系统时，可以设置镜像(Mirror)。
 - 日志文件可占整个硬盘空间的1/5。

SQL Server 属性 (配置) — SUNJIASONG\SUNJIASONG

常规 | 内存 | 处理器 | 安全性 | 连接
服务器设置 | 数据库设置 | 复制 | Active Directory

设置

重建索引时使用的默认索引填充因子 (%):

 ☐ 固定(F) 0  100%

备份/还原

 指定 SQL Server 在读取磁带时应等待的时间。

超时期限: ☒ 无限期等待(W) ☐ 试一次后退出(T) ☐ 试(Y): 分钟

默认的备份媒体保持期 (天) (B):

故障还原

 恢复间隔 (分钟) (R)

新数据库默认位置

 默认数据目录(E): ...

默认日志目录(L): ...

☒ 配置值(C) ☐ 运行值(V)

确定 取消 帮助